# Supplementary Appendix

TreeFlow: Probabilistic Modelling and Automatic Differentiation for Phylogenetics

Swanepoel et al.

## Model specification

TreeFlow provides two ways to specify phylogenetic models. For standard phylogenetic analyses, models can be defined using a YAML-based model definition format that is used as input to TreeFlow's command line interfaces. For more flexible model specification, the Python developer API allows users to directly compose TensorFlow Probability distributions into a joint distribution object.

### YAML model definition format

The YAML model definition format provides a concise, text-based specification of standard phylogenetic models. The format is a nested dictionary with keys at various levels corresponding to model components (tree prior, substitution model, clock model, site model), their parameters, and the prior distributions on those parameters. Figure S1 shows an example YAML model definition for a phylogenetic analysis of influenza sequences, specifying a coalescent tree prior, strict molecular clock, discretized Gamma site rate model, and GTR substitution model with independent Gamma priors on the relative substitution rates.

### Python API model specification

For models that go beyond the standard form supported by the YAML format, the Python developer API provides direct access to TensorFlow Probability's joint distribution framework. This allows users to compose any combination of distributions and transformations into a probabilistic graphical model. Figure S2 shows an example of this flexibility: the code specifies a phylogenetic model for the carnivores dataset, with highlighted lines showing the small changes required to extend the model from a single transition-transversion ratio (kappa) to a per-lineage kappa parameter. This extension, which leverages TensorFlow's vectorization and broadcasting functionality, would be difficult to express in a fixed model definition format.

For further examples and a detailed tutorial on using TreeFlow's Python API and command line interfaces, see the TreeFlow documentation at `https://treeflow.readthedocs.io`.

# Choice of variational approximation family

TreeFlow provides three variational approximation families for fixed-topology models. The *mean field* family treats every coordinate of the (unconstrained) parameter vector as independent. The *full rank* family places a full covariance over the whole parameter vector, including every node height. The *root full rank* family, used for the analyses in the main text, places a full covariance over the substitution, clock and tree-prior parameters together with the tree's root height, and treats the remaining node-height ratios as independent.

The motivation for the third family is that the two conventional choices fail in opposite directions on these models. The strongest posterior correlations in a time-tree analysis are between the parameters that jointly determine the scale of the tree — the clock rate, the tree prior's rate parameter and the root height — and a mean-field approximation cannot represent them at all, so it underestimates the marginal variance of exactly those quantities. A full-rank approximation can represent them, but it must also fit a covariance over all $n-1$ node heights, which is $O(n^2)$ parameters for a problem whose useful correlation structure is concentrated in a handful of coordinates; in practice its optimisation is slower to settle and its fitted marginals for the tree height and clock rate are both biased and over-dispersed. Restricting the covariance block to the shared parameters plus the root height keeps the correlations that matter while leaving the number of variational parameters linear in the number of taxa.

Figures S3 and S4 show the fitted posterior marginals for one run of each family on the carnivores and influenza datasets respectively, against the same BEAST 2 reference posterior. Tables S1 and S2 give the corresponding evidence lower bound and wall-clock runtime. ELBO values are comparable between approximations fitted to the same dataset, but not between datasets: the log likelihood is a sum over alignment patterns, of which the carnivores alignment has substantially more than the influenza alignment despite having far fewer taxa.

| Approximation | ELBO | Runtime (minutes) |
|---|---|---|
| Mean field | -193,601 | 14.4 |
| Full rank | -193,611 | 13.7 |
| Root full rank | -193,600 | 13.9 |

Table S1: Evidence lower bound and wall-clock runtime for each variational approximation family on the carnivores dataset, for the runs shown in Figure S3. Each run is 60,000 optimisation iterations.

# Parameters that grow with the tree

The root full rank family confines its covariance block to parameters that are shared across the whole tree. A model may also have parameters that are replicated per branch — an uncorrelated relaxed clock, or the per-lineage transition-transversion ratio of the carnivores model in Figure S2 — whose dimension grows with the tree in the same way the node heights do. Including these in the covariance block would restore the $O(n^2)$ cost the family exists to

| Approximation | ELBO | Runtime (minutes) |
|---|---|---|
| Mean field | -21,561 | 59.3 |
| Full rank | -21,815 | 66.0 |
| Root full rank | -21,563 | 60.1 |

Table S2: Evidence lower bound and wall-clock runtime for each variational approximation family on the influenza (H3N2) dataset, for the runs shown in Figure S4. Each run is 60,000 optimisation iterations.

avoid, so TreeFlow's approximation constructor takes a `mean_field_vars` argument naming the variables to exclude from the block; they are then approximated independently, alongside the node-height ratios. Any posterior correlation between such a parameter and the rest of the model — for instance between a branch's rate and the heights of the nodes it separates — is therefore not captured by this family, and remains a target for future work.

```yaml
clock:
  strict:
    clock_rate:
      lognormal:
        loc: -2.0
        scale: 2.0
site:
  discrete_gamma:
    category_count: 4
    site_gamma_shape:
      lognormal:
        loc: 0.0
        scale: 1.0
substitution:
  gtr_rel:
    frequencies:
      dirichlet:
        concentration:
        - 2.0
        - 2.0
        - 2.0
        - 2.0
    rate_ac:
      gamma:
        concentration: 0.05
        rate: 0.05
    rate_ag:
      gamma:
        concentration: 0.05
        rate: 0.05
    rate_at:
      gamma:
        concentration: 0.05
        rate: 0.05
    rate_cg:
      gamma:
        concentration: 0.05
        rate: 0.05
    rate_gt:
      gamma:
        concentration: 0.05
        rate: 0.05
tree:
  coalescent:
    pop_size:
      lognormal:
        loc: 1.0
        scale: 1.5
```

Figure S1: YAML model definition for the analysis of the influenza dataset. The top-level keys specify the clock model, site rate model, substitution model, and tree prior. Each model component's parameters are given prior distributions from the standard distributions available in TensorFlow Probability.

```
site_category_count = 4
pattern_counts = alignment.get_weights_tensor()
subst_model = HKY()

def build_sequence_dist(tree, kappa, frequencies, site_gamma_shape):
    unrooted_tree = tree.get_unrooted_tree()
    site_rate_distribution = DiscretizedDistribution(
        category_count=site_category_count,
        distribution=Gamma(
            concentration=site_gamma_shape,
            rate=site_gamma_shape
        ),
    )
    transition_probs_tree = get_transition_probabilities_tree(
        unrooted_tree,
        subst_model,
        rate_categories=site_rate_distribution.normalised_support,
        frequencies=frequencies,
        frequencies=tf.broadcast_to(
            tf.expand_dims(frequencies, -2),
            kappa.shape + (4,)
        ),
        kappa=kappa
    )
    return SampleWeighted(
        DiscreteParameterMixture(
            site_rate_distribution,
            LeafCTMC(
                transition_probs_tree,
                expand_dims(frequencies, -2),
            ),
        ),
        sample_shape=alignment.site_count,
        weights=pattern_counts
    )

model = JointDistributionNamed(dict(
    birth_rate=LogNormal(c(1.0), c(1.5)),
    tree=lambda birth_rate: Yule(tree.taxon_count, birth_rate),
    kappa=LogNormal(c(0.0), c(2.0)),
    kappa=Sample(
        LogNormal(c(0.0), c(2.0)),
        tree.branch_lengths.shape
    ),
    site_gamma_shape=LogNormal(c(0.0), c(1.0)),
    frequencies=Dirichlet(c([2.0, 2.0, 2.0, 2.0])),
    sequences=build_sequence_dist
))
```

Figure S2: Code to specify models for the carnivores analysis using TreeFlow's Python API. Highlighted lines show changes (from previous line) to add kappa variation across lineages. The `Sample` distribution is used to make the kappa prior an independent identically-distributed sample for each branch. The frequencies parameter needs to be manually broadcasted over the added kappa dimension.

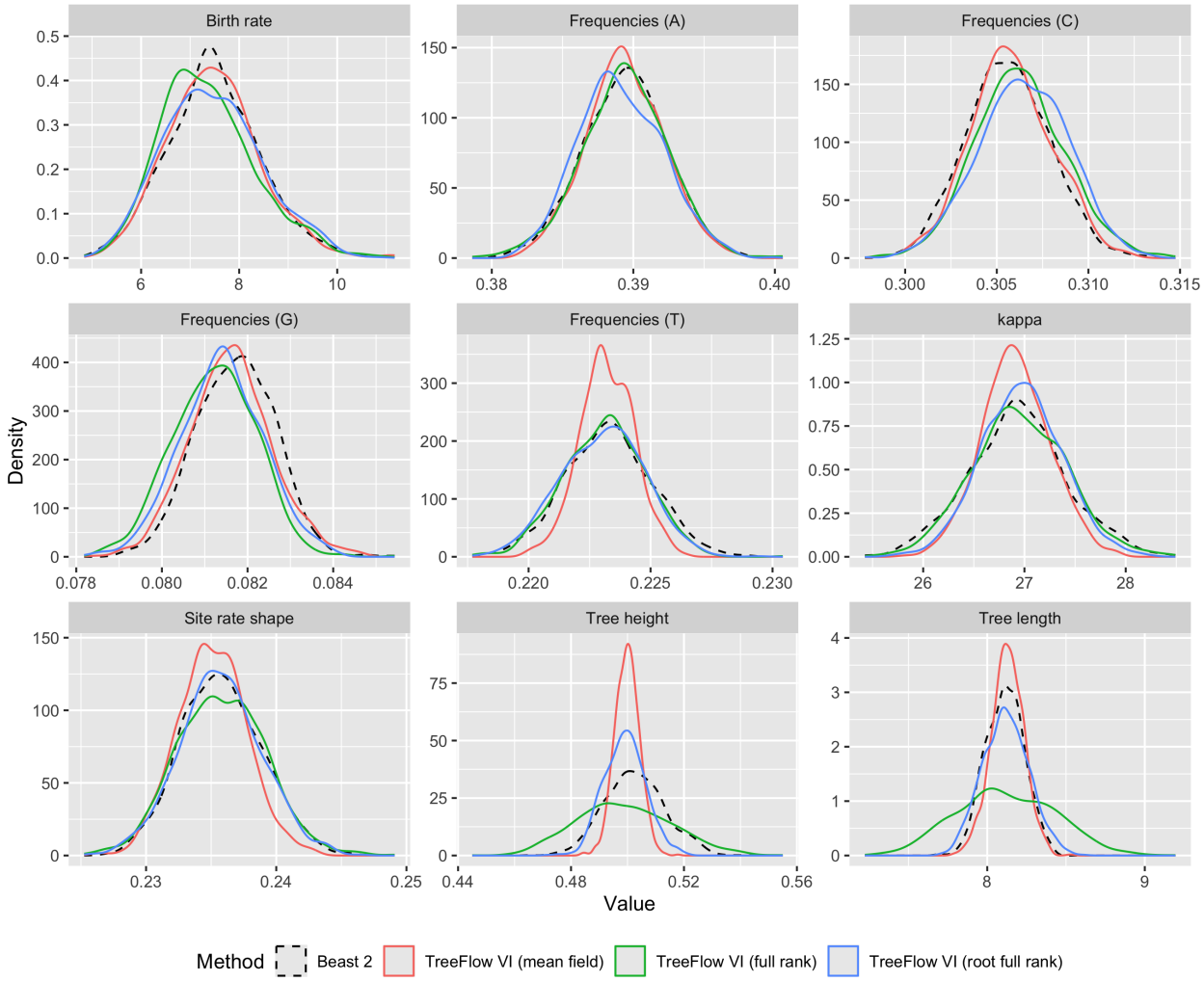


Figure S3: Posterior marginals for the carnivores dataset under each of TreeFlow's three variational approximation families (solid lines), against the BEAST 2 reference posterior (dashed). One variational run per family.

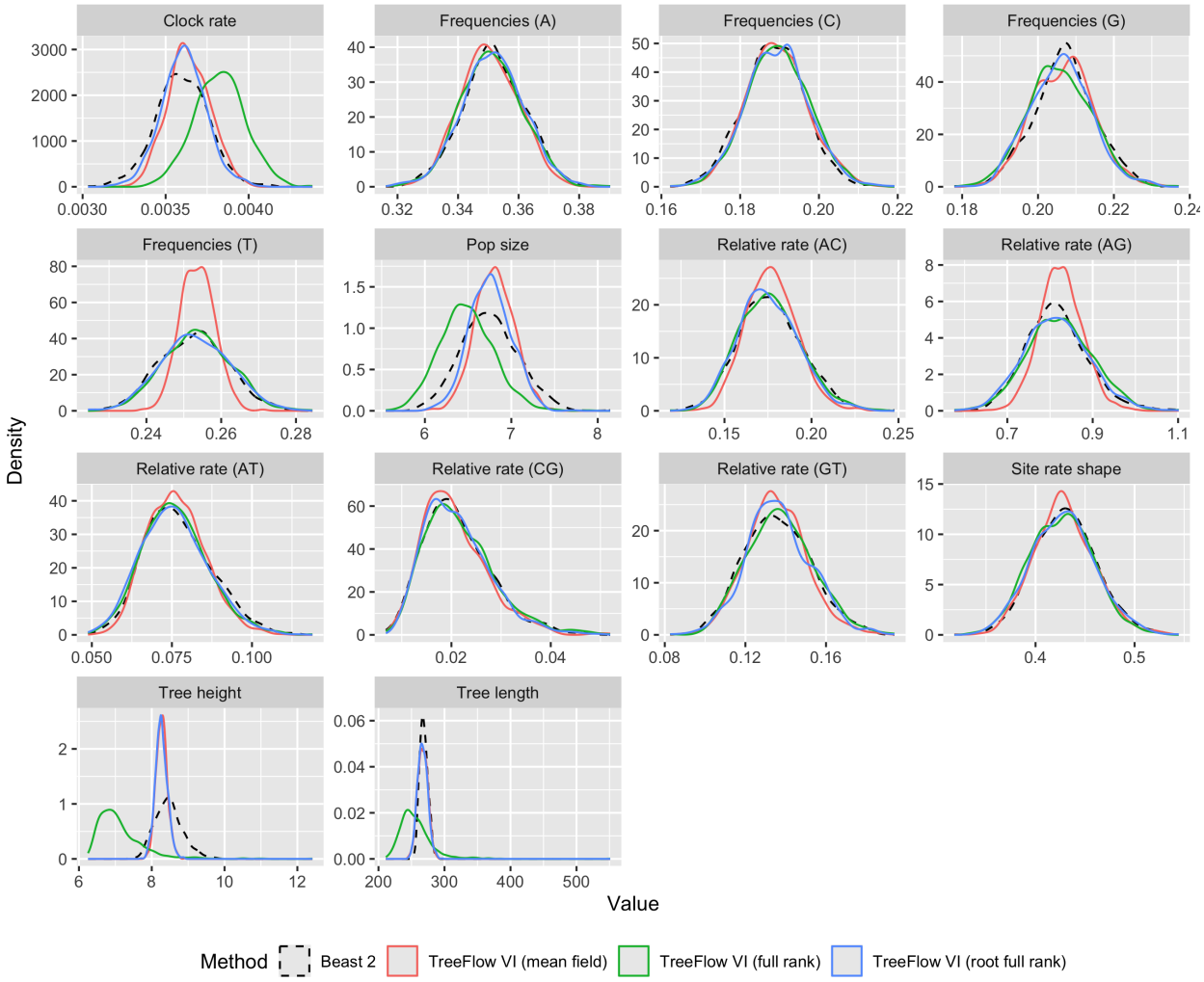


Figure S4: Posterior marginals for the influenza (H3N2) dataset under each of TreeFlow's three variational approximation families (solid lines), against the BEAST 2 reference posterior (dashed). One variational run per family.